Trabajo: Optimización II

Asier Bóveda Ander Escaray

Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Grado en Ciencia de Datos

Mayo 2025

Índice

1. Introducción **2**

2. Asignación de municipios a sedes y su impacto en la disponibilidad **3**

3. Vecindario **5**

4. Elección del algoritmo **6**

5. Explicación del código **6**

5.1. Generación de una solución aleatoria 7

5.2. Iteraciones de mejora 7

6. Optimización de Parámetros **8**

7. Análisis computacional y temporal **10**

8. Estudio de una muestra de los soluciones obtenidas **11**

9. Estudio de la mejor solución obtenida **11**

10. Conclusiones y discusión **14**

1. Introducción

La localización óptima de recursos de emergencia es un problema crucial en la planificación sanitaria. En este trabajo se aborda el problema de la ubicación de ambulancias en Navarra, con el objetivo de mejorar la cobertura y eficiencia del servicio de urgencias.

Se plantea la necesidad de desplegar ambulancias de forma que se minimizen tanto los tiempos de respuesta ante emergencias como el coste económico asociado. El problema incluye múltiples restricciones relacionadas con la calidad del servicio:

- Al menos el 98 % de la población debe estar cubierta por una ambulancia situada a menos de 40 kilómetros.
- Al menos el 80 % de la población debe estar cubierta por una ambulancia situada a menos de 20 kilómetros.
- En cada sede, la probabilidad de disponibilidad inmediata de una ambulancia debe ser al menos del 80 %.

Para estimar esta probabilidad de disponibilidad, se considera que una ambulancia está ocupada durante un tiempo medio estimado como:

$$T_{\text{medio}} = \frac{d(\text{sede}, \text{emergencia}) + d(\text{emergencia}, \text{hospital})}{80} + 1{,}5$$

Donde las distancias están en kilómetros y la velocidad media se supone de 80 km/h. El término fijo de 1.5 horas representa el tiempo de activación, atención *in situ* y traslado del paciente. Como hospital de referencia se toma Pamplona.

El número esperado de salidas diarias por sede se estima como:

$$\text{Nº Medio de salidas} = 3 \cdot \left(\frac{\text{Tamaño poblacion cubierta}}{100.000} \right)$$

Multiplicando las salidas por el tiempo medio de ocupación se obtiene la carga de trabajo diaria total de la sede, medida en horas:

$$\text{carga_trabajo} = \text{Nº Medio de salidas} \cdot T_{\text{medio}}$$

Si una sede dispone de n ambulancias, se reparte esta carga entre ellas. La probabilidad de que una ambulancia individual esté libre se aproxima como:

$$p_{\text{libre}} = 1 - \left(\frac{\text{carga_trabajo}}{24 \cdot n} \right)$$

La probabilidad de que al menos una ambulancia esté disponible se calcula como el complemento de que todas estén ocupadas simultáneamente:

$$P_{\text{disponible}} = 1 - (1 - p_{\text{libre}})^n$$

Además de cumplir estas restricciones, se busca minimizar una función objetivo que penaliza el número de sedes abiertas, la cantidad de ambulancias desplegadas y la distancia total ponderada por población.

Para resolver este problema, se ha desarrollado un algoritmo heurístico que combina *simulated annealing* y *adaptative large neighboorhood search*, a lo largo de este informe se explicará la elección del vecindario, la función fitness, y algunas particularidades para la resolución de este ejercicio en concreto.

2. Asignación de municipios a sedes y su impacto en la disponibilidad

En este trabajo se asigna cada municipio a la sede de ambulancias más cercana, con el objetivo de minimizar los tiempos de respuesta. No obstante, una alternativa sería distribuir los municipios teniendo en cuenta no solo la cercanía, sino también el impacto que tienen sobre la probabilidad de que una sede necesite más ambulancias. En particular, podría pensarse en asignar ciertos municipios a sedes más lejanas si con ello se evita tener que añadir una ambulancia adicional en una sede ya saturada. Queremos estudiar cuánto puede aumentar la población asignada a una sede sin que la probabilidad de disponibilidad de ambulancias se vea afectada significativamente. En particular, buscamos una cota superior para el cambio en la probabilidad de disponibilidad al asignar una cantidad adicional de población Δp a una sede con n ambulancias.

Recordemos que:

- La carga diaria de trabajo en horas es $c(p) = \dfrac{3t}{100{,}000} \cdot p$, siendo t el tiempo medio de atención por salida. Aquí, estamos suponiendo que al añadir dicha población, la media de distancia no varía. Al ser la media de varias poblaciones vamos a despreciar dicho incremento para simplificar el problema.
- La carga por ambulancia es $c(p)/n$.
- La probabilidad de que una ambulancia esté libre es $p_{\text{libre}} = 1 - \dfrac{c(p)}{24n}$.
- La probabilidad de que al menos una ambulancia esté disponible en la sede es:

$$P(p) = 1 - (1 - p_{\text{libre}})^n = 1 - \left(\frac{c(p)}{24n}\right)^n$$

La variación de P al aumentar la población en Δp se puede acotar . Si $\varepsilon > 0$ es el máximo cambio tolerado de la probabilidad, entonces:

$$\Delta P \approx \left|\frac{dP}{dp}\right| \cdot \Delta p \leq \varepsilon$$

Derivamos $P(p)$ respecto a p:

$$\left|\frac{dP}{dp}\right| = \left(\frac{3tp}{2{,}400{,}000n}\right)^{n-1} \cdot \frac{3t}{2{,}400{,}000}$$

Entonces, una cota para la población adicional sin superar ε es:

$$\Delta p \leq \varepsilon \cdot \left(\frac{2{,}400{,}000}{3t}\right)^{n} \cdot \left(\frac{p}{n}\right)^{n-1}$$

Esto significa que se relizamos un gran cambio en la población no repercute mucho en la probabilidad. Esta claro que si la ambulancia es uno la probabilidad es más sensible a cambios, siendo una relación lineal. Sin embargo, el coeficiente $800000/t$ es grande, siendo t el tiempo de atención por salida en horas. Vamos a poner un ejemplo para que se entienda mejor:

Ejemplo numérico: Si consideramos los valores:

- $t = 2{,}5$ horas (tiempo medio de atención por emergencia)
- $n = 1$ ambulancia (con una ambulancia la probabibilidad es más sensible a cambios en la población, al no tener ningun término elevado)
- $p = 10{,}000$ habitantes
- $\varepsilon = 0{,}01$

Entonces:

$$\Delta p \leq 0{,}01 \cdot \left(\frac{2{,}400{,}000}{7{,}5}\right) \cdot \left(\frac{2{,}400{,}000}{25{,}000}\right)^{0} = 0{,}01 \cdot 320{,}000 \cdot 1 = \boxed{3{,}200}$$

Esto significa que si una sede tiene 1 ambulancia, un tiempo medio de 2.5 horas por emergencia y una población de 10,000 habitantes, entonces se puede aumentar la población asignada en hasta 3,200 personas sin que la probabilidad de disponibilidad se reduzca más de un 1 %. Este valor es alto teniendo en cuenta que como es la distribución de la población Navarra.

Y esto es lo que nos ha llevado a simplificar el problema asignando la sede más cercana a cada municipio. Como ya hemos explicado, existe la posibilidad de cambiar asignaciones baje el número de ambulancias. Sin embargo, el número de asignaciones que hay cambiar es muy elevado, lo que va en contra del último término de la función objetivo, que trata de minimizar la distancia media de cada sede a la población total. Además, la complejidad del problema baja de forma exponencial. Previamente nos enfrentabamos a buscar las sedes mejor situadas y además, asignar de forma que las ambulancias y las asignaciones sean óptimas. Ahora el problema es mucho más simple. Puesto que simplemente posicionando las sedes, calcularemos de forma automática las asignaciones (cada municipio a la sede más cercana) y posteriormente, calcularemos las ambulancias necesarias teniendo en cuenta la población total asignada a dicha sede.

Concluimos este apartado definiendo nuestro espacio de soluciones. Este es una lista ordenada de variables binarias con la misma cardinalidad que el número de municipios. De esta forma, si un municipio es sede, tendrá un valor de 1 en la posición de la lista. El espacio de soluciones es el conjunto de todas las posibles opciones de dicha lista. Mientras que el espacio de soluciones factibles, el subconjunto de dichas soluciones que verifican las 2 primeras restricciones. Ya que la restricción de la probabilidad de que una ambulancia esté libre se asegurará de forma automática. Ahora debemos encontrar un vecindario que nos permita llegar a todo el espacio de soluciones factibles.

3. Vecindario

El vecindario que hemos aplicado en nuestro algoritmo está basado en el *adaptative large neighboorhood search*, donde existen diferentes vecindarios que pueden ser elegidos al azar. Sea $\mathcal{M} = \{1, 2, \ldots, m\}$ el conjunto de todos los municipios de Navarra. Cada solución factible se representa como un conjunto

$$s \subseteq \mathcal{M},$$

donde $i \in s$ indica que existe una sede de ambulancias en el municipio $i \in \{0, 1\}$. Necestiamos un vecindario que nos permita además de modificar sedes modificar el número de sedes, es por ello que hemos generado un vecindario complementando varios:

Vecindarios elementales. A partir de una solución s, definimos tres tipos de movimiento que generan vecinos s':

$$\begin{aligned} N_{\text{add}}(s) &= \big\{\, s' = s \cup \{j\} \;\big|\; j \in \mathcal{M} \setminus s \big\}, \\ N_{\text{remove}}(s) &= \big\{\, s' = s \setminus \{i\} \;\big|\; i \in s \big\}, \\ N_{\text{move}}(s) &= \big\{\, s' = (s \setminus \{i\}) \cup \{j\} \;\big|\; i \in s,\ j \in \mathcal{M} \setminus s \big\}. \end{aligned}$$

Vecindario total. El vecindario que exploramos en cada iteración de la metaheurística es la unión de estos tres sub-vecindarios:

$$N(s) \;=\; N_{\text{add}}(s) \;\cup\; N_{\text{remove}}(s) \;\cup\; N_{\text{move}}(s).$$

Elección aleatoria de movimientos. En cada paso generamos un vecino $s' \in N(s)$ de la forma siguiente:

1. Elegimos al azar uno de los tres tipos de movimiento: `add`, `remove` o `move`.
2. Aplicamos dicho movimiento seleccionando uniformemente el municipio o sede correspondiente.
3. Recalculamos las asignaciones y el número de ambulancias que debemos asignar.

Probabilidades adaptativas. Como en el esquema del LNS, la probabilidad de que se entre a cada vecindario varía dependiendo de lo útil que este sea para mejorar la función

objetivo. El método metaheurístico en el que nos hemos basado es el Simulated Annealing, que no realiza una búsqueda intensiva del vecindario. Sin embargo, hemos decidido reducir la probabilidad de volver a entrar en ese sub-vecindario si la solución obtenida al azar en ese vecindario es peor que la anterior.

En este punto queremos justificar por que no comprobamos las restricciones de que un mínimo del 80 % y 98 % de la población este a menos de 20 y 40 kilómetros, respectivamente. Y es que, debido al término

$$\sum Pi \cdot D_{ij}$$

de la función objetivo, estas restricciones estan ponderadas negativamente. Es más, mediante el estudio empírico de decenas de ejecuciones de este algoritmo en ninguna solución ha supuesto un problema las restricciones. Esto es debido a que se tratan de unas restricciones muy laxas y que, como ya hemos comentado, estan ponderadas ya de por sí por la función objetivo. Este hecho se puede visualizar mejor en los gráficos del mapa interactivo.

4. Elección del algoritmo

El algoritmo que hemos elegido para la resolución del problema es un *Simulated Annealing*. Esto se debe principalmente a tres puntos. En primer lugar, el problema que se nos presenta es un problema multimodal. La cantidad de máximos locales existentes en el espacio de soluciones es muy alto, y por lo tanto es muy fácil quedar atascado en una de esas soluciones. Es por ello, por lo que el método de escape mediante la Regla de Metropolis es muy útil.

Por otro lado, el tamaño del problema no es excesivamente grande. Esto se puede apreciar en la solución óptima obtenida que comentaremos más adelante cuenta tan solo con 12 sedes. Esto combinado con la velocidad de los cálculos para cambiar de vecindario hace que la exploración de las soluciones se muy grande, aún en periodos cortos de tiempos. Y por último, nos basamos en la razón empírica para decirnos por este algoritmo. Ya que es el que mejores resultados ha dado entre Grasp, VNS y Tabu Search.

5. Explicación del código

El script implementa un algoritmo de Simulated Annealing para la localización de sedes y la asignación de ambulancias en Navarra. Primero definimos la clase `Solucion`, que se instancia con un único parámetro: la lista de códigos de municipios en los que se abrirá una sede de ambulancias. A partir de esa lista, el constructor realiza automáticamente:

- La asignación de cada municipio a su sede más cercana.
- El cálculo del número mínimo de ambulancias por sede para garantizar la disponibilidad (restricción R3).

- La evaluación de la función objetivo, que engloba el coste de apertura de sedes, el número de ambulancias y la distancia ponderada por población.

5.1. Generación de una solución aleatoria

La función `generar_sol_aleatoria()` construye iterativamente una solución inicial que cumpla las restricciones R1 y R2:

Bucle de generación. Se repite hasta encontrar una solución válida:

1. Se extrae aleatoriamente un número de sedes

$$n_{\text{sedes}} \sim \mathcal{U}\{\,10,\ 11,\ \ldots,\ |\mathcal{M}| - 100\},$$

   donde $|\mathcal{M}|$ es el número total de municipios.

2. Se escoge un subconjunto $s \subseteq \mathcal{M}$ de tamaño n_{sedes} mediante muestreo uniforme (`random.sample`).

3. Se instancia la solución

$$\texttt{sol} = \texttt{Solucion}(s).$$

5.2. Iteraciones de mejora

El flujo principal del programa es el siguiente:

1. Inicialización de parámetros:

$$T \leftarrow 500, \quad T_{\text{mín}} \leftarrow 0{,}00000001, \quad \alpha \leftarrow 0{,}95, \quad \text{num_iters} \leftarrow 75.$$

   Además, se genera una semilla pseudoaleatoria `seed = random.randint(0,1000000)`, y se fija con `random.seed(seed)`. Esto hace que sea posible recrear la ejecución exactamente igual.

2. Inicialización de probabilidades de movimiento:

$$p_{\text{add}} = p_{\text{remove}} = p_{\text{move}} = \tfrac{1}{3}.$$

3. Solución inicial:

$$\texttt{sol} = \texttt{generar_sol_aleatoria()}, \quad f_{\text{obj}} = \texttt{sol._funcion_objetivo()}.$$

4. Bucle de Simulated Annealing:
   - *Mientras* $T > T_{\text{mín}}$:
     - *Para* cada una de las num_iters iteraciones:
       - `(sol_cand,mov)` = `generar_vecino(sol)`
       - f_{cand} = `sol_cand._funcion_objetivo()`

- *Si* $f_{\text{cand}} < f_{\text{obj}}$, *aceptar* el vecino:

$$sol \leftarrow sol_{\text{cand}}, \quad f_{\text{obj}} \leftarrow f_{\text{cand}}.$$

- *Sino*, calcular la probabilidad de aceptación $\exp\big((f_{\text{obj}} - f_{\text{cand}})/T\big)$; si se rechaza, *ajustar* las probabilidades $\{p_{\text{add}}, p_{\text{remove}}, p_{\text{move}}\}$:

$$p_{\text{mov}} \leftarrow \text{máx}(0, p_{\text{mov}} - c), \quad p_{\text{otras}} \leftarrow p_{\text{otras}} + \tfrac{c}{2},$$

  y normalizar de modo que sumen 1.
- Añadir f_{obj} a la lista `fobj_values`.

- Actualizar temperatura:

$$T \leftarrow \alpha\, T.$$

5. Salida. Al terminar, se guarda la mejor solución encontrada y se exporta a un archivo Excel.

6. Optimización de Parámetros

Para que el algoritmo de *Simulated Annealing* funcione de manera eficiente, es importante ajustar adecuadamente sus hiperparámetros. Estos valores influyen en el equilibrio entre exploración y explotación, y determinan tanto la calidad de las soluciones como el tiempo de cómputo necesario. En esta sección se recoge un estudio empírico de sensibilidad sobre los siguientes parámetros:

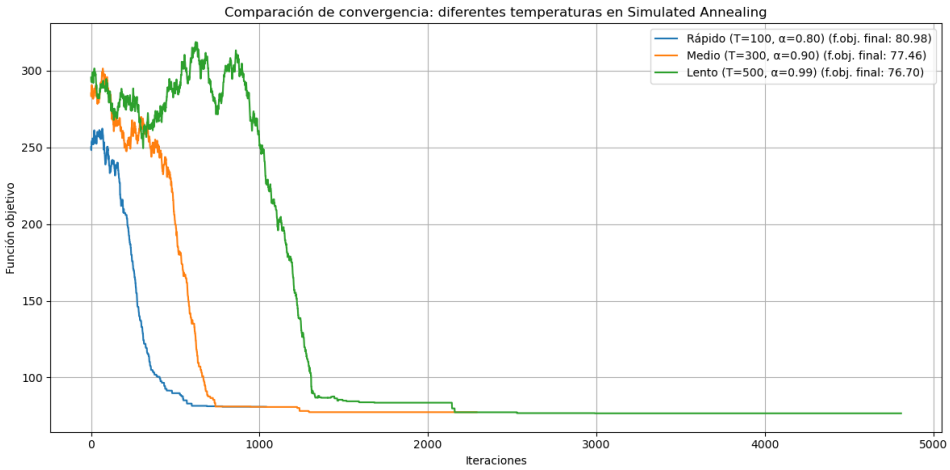
$$T_0, \quad T_{\text{mín}}, \quad \alpha, \quad c, \quad \text{num_iters}.$$

La metodología seguida ha consistido en modificar uno de los parámetros cada vez, manteniendo los demás constantes. En cada configuración se ha ejecutado el algoritmo varias veces, anotando el valor final medio de la función objetivo y el tiempo empleado. Esto permite detectar qué combinaciones ofrecen un buen equilibrio entre precisión y coste computacional.

Uno de los parámetros que primero se ha analizado es c, que se utiliza para ajustar dinámicamente la probabilidad de elegir cada vecindario. Según cómo se comporten los vecindarios (es decir, si suelen producir mejoras o no), este parámetro aumenta o disminuye, y luego se normalizan las probabilidades. Aunque los resultados muestran que el valor concreto de c no altera drásticamente la convergencia, hemos observado que si se elige un valor muy grande, se tiende a penalizar excesivamente algunos movimientos, como añadir o quitar sedes, que son precisamente los que más variedad introducen en el espacio de búsqueda. Por ese motivo, se ha optado por mantener un valor bajo, en concreto $c = 0{,}001$, ya que permite mantener activa la diversidad de movimientos a lo largo de toda la ejecución.

Una vez fijado ese valor, el siguiente paso ha sido estudiar el efecto de la temperatura inicial T_0 y del factor de enfriamiento α, que controlan la probabilidad de aceptar soluciones peores y, por tanto, la capacidad del algoritmo de escapar de óptimos locales. Se han comparado tres configuraciones diferentes:

En el grafico se muestra la evolución de la función objetivo a lo largo de las iteraciones para cada una de estas configuraciones. Todas las ejecuciones se han realizado con la misma semilla para poder comparar los resultados de forma justa.



Tal y como se observa en el gráfico, el enfriamiento rápido permite una convergencia muy veloz, pero conduce a una solución peor ($f = 80{,}98$), lo que sugiere que el algoritmo se ha quedado atrapado en un óptimo local. En cambio, la configuración con enfriamiento lento explora durante más tiempo y consigue una mejor solución final ($f = 76{,}70$), aunque requiere muchas más iteraciones. El caso intermedio representa un compromiso aceptable, alcanzando un valor de $f = 77{,}46$ en un tiempo razonable.

Estos resultados refuerzan la idea de que una temperatura inicial alta, junto con un enfriamiento gradual, da al algoritmo más margen para explorar el espacio de soluciones antes de converger, lo cual suele ser beneficioso en problemas con muchos óptimos locales.

Con base en este análisis, se ha optado por los siguientes valores como configuración final del algoritmo:

- Temperatura inicial: $T_0 = 500$
- Temperatura mínima: $T_{\min} = 0{,}00000001$
- Factor de enfriamiento: $\alpha = 0{,}95$
- Constante de adaptación de probabilidades: $c = 0{,}001$
- Número de iteraciones por temperatura: 75

Esta elección responde a un equilibrio entre calidad de las soluciones y tiempo de ejecución. Aunque otras configuraciones podrían acelerar la convergencia, suelen implicar una pérdida de calidad. No obstante, si fuese necesario reducir el tiempo de cálculo —por ejemplo, en aplicaciones en tiempo real—, sería posible disminuir T_0 a valores como 100 o 150 sin que la solución se resienta demasiado, gracias a la estabilidad del algoritmo.

En conjunto, esta calibración permite obtener soluciones robustas y eficientes, adaptadas a la estructura concreta del problema que se está resolviendo.

7. Análisis computacional y temporal

Desde el principio hemos querido que nuestro algoritmo se amolde a diferentes posibilidades. Es por ello que, dependiendo de los parámetros utilizados, la complejidad del algoritmo puede variar ligeramente. Vamos a comenzar comentando la complejidad del cambio de vecindario. Sea cual sea el vecindario escogido al azar, la complejidad del cambio tiene un orden de $O(n)$, siendo n el número de municipios del problema.

Los cálculos realizados durante este cambio son los siguientes: se ejecuta una permutación, adición o eliminación de una sede al azar. A continuación, se recorre la lista de municipios para reasignar cada uno a su sede más cercana (en caso de que hayan cambiado), y se calcula la población total asignada a cada sede. Posteriormente, se calcula el número de ambulancias necesarias en cada sede en función del tiempo medio de respuesta y la carga estimada. Finalmente, se vuelve a recorrer la lista de municipios para evaluar el valor de la función objetivo.

Aunque se realizan varios bucles sobre la lista de municipios, cada uno de estos pasos es computacionalmente ligero, lo que permite realizar exploraciones amplias del espacio de soluciones en tiempos reducidos.

En cuanto al número de iteraciones en las que se ejecuta este cambio de vecindario, el esquema de enfriamiento adoptado se basa en los siguientes parámetros:

- Temperatura inicial: $T_0 = 500$
- Temperatura mínima: $T_{\min} = 0{,}001$
- Factor de enfriamiento: $\alpha = 0{,}95$
- Iteraciones por nivel de temperatura: 75

El número de niveles de temperatura necesarios para alcanzar $T_{\min}$ se calcula mediante la fórmula:

$$k = \left\lceil \frac{\log(T_{\min}/T_0)}{\log(\alpha)} \right\rceil = \left\lceil \frac{\log(0{,}001/500)}{\log(0{,}95)} \right\rceil = \left\lceil \frac{-5{,}698}{\log(0{,}95)} \right\rceil = \left\lceil \frac{-5{,}698}{-0{,}022} \right\rceil = 259$$

Por tanto, el número total de iteraciones del algoritmo es:

$$\text{Total de iteraciones} = 259 \cdot 75 = \boxed{19{,}425}$$

En cuanto al rendimiento práctico, las pruebas realizadas indican que una ejecución típica del algoritmo con estos parámetros tarda entre 2 y 3 minutos en completarse, lo cual resulta aceptable dado el tamaño del problema, que incluye más de 250 municipios.

Además, el algoritmo muestra una buena capacidad de convergencia: en muchos casos, se alcanzan soluciones de alta calidad en las primeras 10000-15000 iteraciones, lo que equivale aproximadamente al segundo minuto de ejecución. Esto sugiere que, si se requiere una respuesta más rápida, el algoritmo puede adaptarse fácilmente incrementando $T_{\min}$ o

reduciendo el número de iteraciones por nivel, manteniendo una calidad razonable en las soluciones obtenidas.

En definitiva, consideramos que la configuración adoptada ofrece un equilibrio adecuado entre exploración y tiempo de ejecución, permitiendo asegurar soluciones cercanas al óptimo en un tiempo razonable.

8. Estudio de una muestra de los soluciones obtenidas

Para evaluar la estabilidad del algoritmo con los parámetros seleccionados, se realizaron 30 ejecuciones independientes del Simulated Annealing con distintas semillas. El objetivo era analizar cómo se distribuyen los valores de la función objetivo final, y si existe consistencia en la calidad de las soluciones alcanzadas.

Los resultados obtenidos muestran una distribución altamente concentrada en torno a dos valores dominantes:

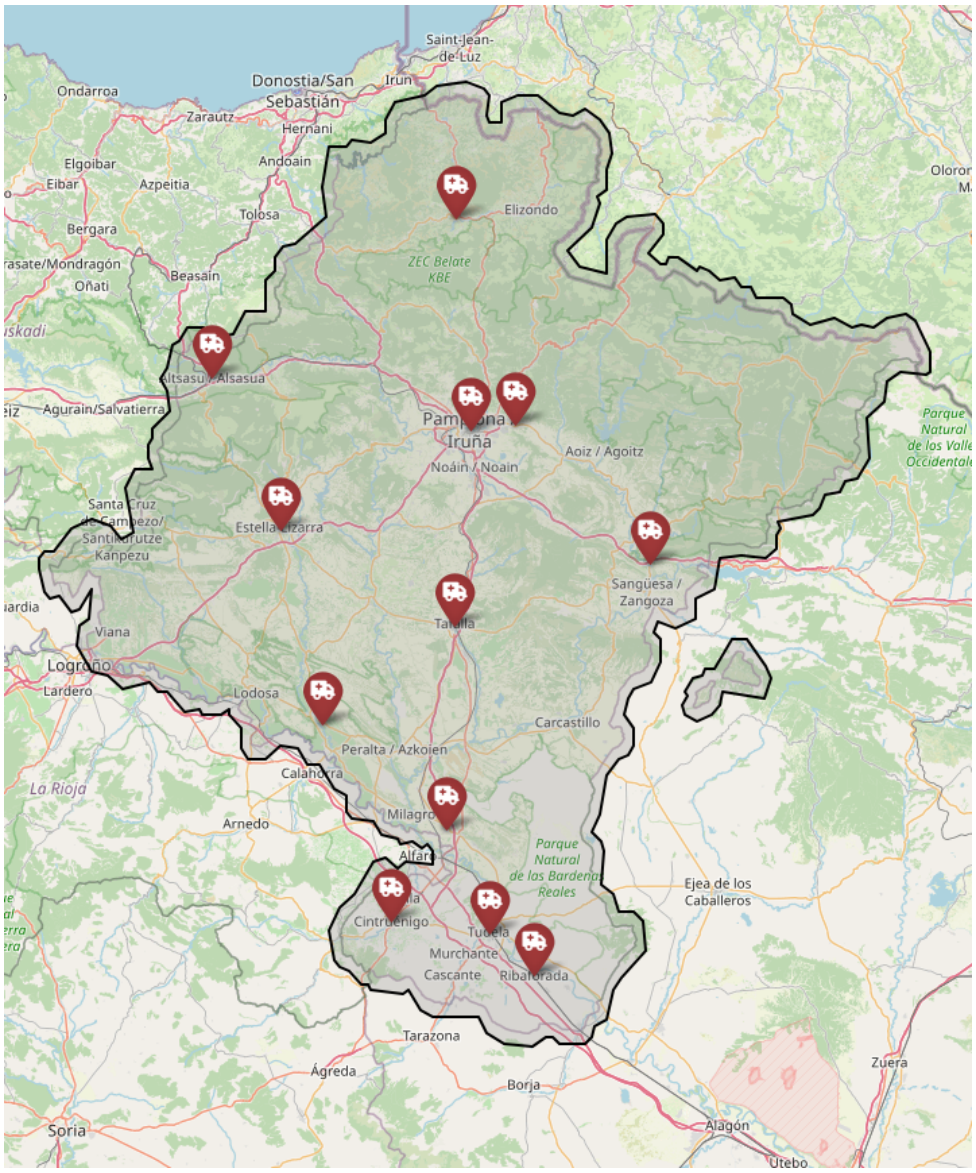
- En 8 de las 30 ejecuciones (26.7 %), el valor final fue exactamente 76,364.
- En 19 ejecuciones (63.3 %), el valor fue 76,374.
- En las 3 ejecuciones restantes (10 %), se obtuvo un valor ligeramente superior de 76,500.

Esto indica que el algoritmo, con la configuración adoptada, es muy estable y tiende a converger sistemáticamente a un subconjunto muy reducido del espacio de soluciones de alta calidad. La mayoría de las soluciones se agrupan en torno a un rango muy estrecho (± 0.2 %), lo que confirma tanto la eficacia como la consistencia del método.

Cabe destacar que las pocas soluciones que alcanzaron el valor 76,500 no son necesariamente malas, sino que pueden representar configuraciones ligeramente distintas en cuanto a distribución de ambulancias, pero que siguen siendo factibles y competitivas.

9. Estudio de la mejor solución obtenida

A continuación presentamos la solución óptima hallada por el algoritmo. En el mapa está trazada la silueta de Navarra y se marcan con un icono de ambulancia las ubicaciones de las 12 sedes seleccionadas:



Las sedes elegidas son las siguientes (todas con una ambulancia salvo Pamplona, que cuenta con dos):

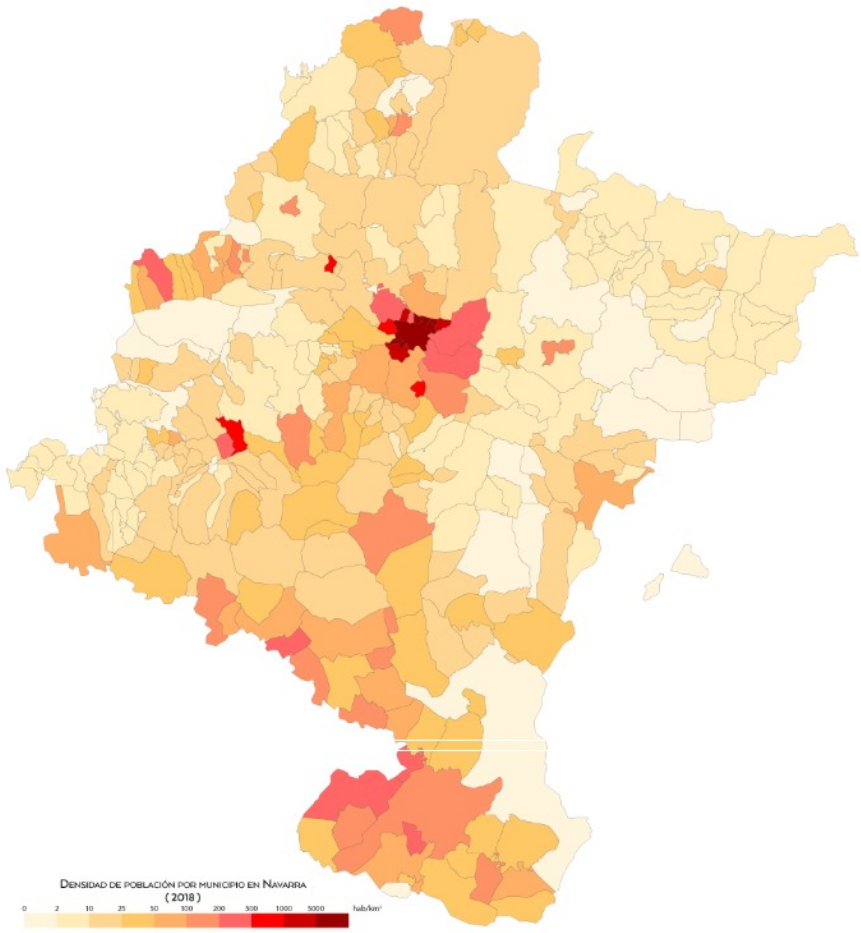
- Altsasu / Alsasua
- Andosilla
- Cadreita
- Cintruénigo
- Valle de Egüés / Eguesibar
- Estella-Lizarra
- Liédena
- **Pamplona / Iruña (2 ambulancias)**

- Ribaforada
- Doneztebe / Santesteban
- Tudela
- Tafalla

La mejor solución encontrada presenta un valor de función objetivo de aproximadamente

$$f_{\text{obj}} = 76{,}365,$$

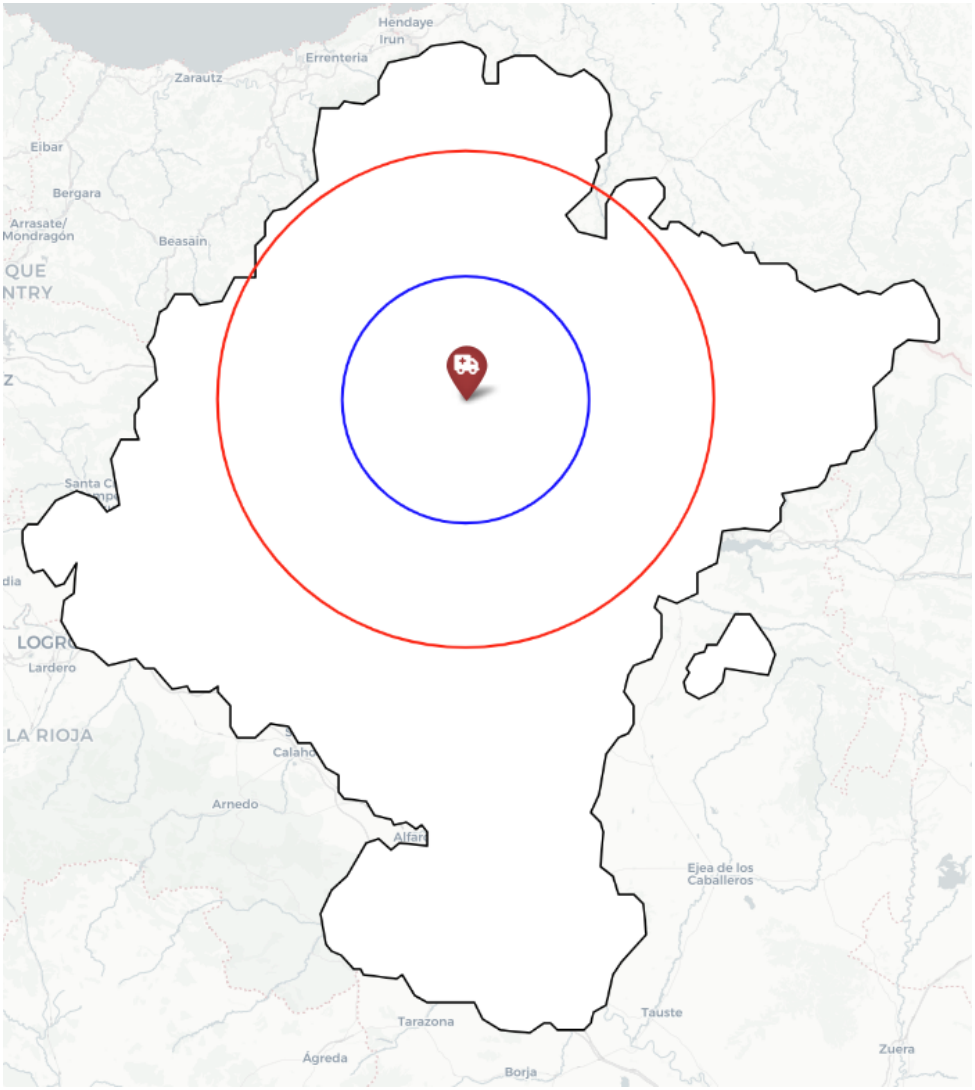
lo que supone una mejora significativa frente a los valores iniciales (alrededor de 280) y demuestra la eficacia del Simulated Annealing con los vecindarios y parámetros elegidos.



Densidad de población por municipio en Navarra (2018). Las áreas más oscuras (rojo) coinciden con los emplazamientos de las sedes.

Como se puede observar, la distribución de las sedes sigue de forma muy estrecha los núcleos de mayor densidad de población, ya que nuestro término de la función objetivo $\sum_j P_j d_{jA}$ pondera los desplazamientos por el número de habitantes de cada municipio.

Para ilustrar gráficamente el área de cobertura de una sede conforme a las restricciones R1 y R2, presentamos a continuación los radios de 20km (azul) y 40km (rojo) centrados en la sede de Pamplona, superpuestos a la silueta de Navarra:



Cobertura de 20km (azul) y 40km (rojo) desde la sede de Pamplona, sobre el contorno de la Comunidad Foral de Navarra.

Para una exploración más detallada —con la posibilidad de activar o desactivar los círculos de cobertura de cada sede, hacer zoom, consultar coordenadas y ver el número exacto de ambulancias— acceda al mapa interactivo en siguiente HTML.

10. Conclusiones y discusión

En este trabajo hemos abordado un problema de optimización complejo, modelado a partir de un caso real de planificación sanitaria: la ubicación óptima de ambulancias en Navarra. A lo largo del proyecto, se ha realizado un modelado matemático , diseñado un espacio de soluciones

bien, y desarrollado un algoritmo basado en *Simulated Annealing* con adaptaciones inspiradas en *Large Neighborhood Search*.

Los resultados obtenidos muestran que el algoritmo es capaz de encontrar soluciones de alta calidad en tiempos razonables, incluso con más de 250 municipios. La estructura del vecindario, junto con el uso de probabilidades adaptativas, ha demostrado ser eficaz para explorar el espacio de soluciones sin caer en óptimos locales prematuramente.

En el apartado de optimización de parámetros, se ha realizado un estudio empírico para ajustar cuidadosamente los valores de T_0, $T_{\min}$, α, c y el número de iteraciones por temperatura. Esta fase ha sido clave para garantizar un buen equilibrio entre tiempo de ejecución y calidad de solución. En particular, se ha comprobado que el algoritmo converge rápidamente, y que, si fuese necesario reducir el tiempo computacional, bastaría con aumentar la temperatura mínima para obtener soluciones razonables en menos tiempo.

En resumen, el algoritmo implementado no solo ha sido efectivo, sino también flexible, y puede adaptarse fácilmente a variaciones del problema, como nuevas restricciones(añadiriamos ponderaciones negativas si no son tan laxas, cambios en la red de municipios o modificaciones en las prioridades del sistema de salud.